

SCUOLA FUTURO LAVORO

AI

Agentic Coding & Context Engineering

Dalle fondamenta agli ecosistemi di agenti

INDICE

Sommario

Introduzione..... 3

PARTE I

Le fondamenta..... 6

01 Account e accessi..... 7

02 Orientarsi nel computer..... 9

03 Il testo puro..... 12

04 La famiglia dei formati..... 14

05 Markdown..... 16

06 Il terminale..... 21

07 Condividere codice ed errori..... 24

DISPENSA DEL CORSO

Introduzione

— A chi si rivolge questa dispensa

Questa dispensa accompagna il corso **AI: Agentic Coding & Context Engineering**. È pensata per chi parte da zero: non serve saper programmare, non serve aver mai aperto un terminale, non serve sapere cosa sia un repository.

Serve solo la disponibilità a fare due cose che all'inizio sembrano scomode:

- **scrivere invece di cliccare**, perché è così che si parla agli strumenti di cui ci occupiamo;
- **guardare cosa succede prima di dire che non funziona**, perché è da lì che si impara.

Il corso è rivolto a studenti, professionisti, creativi e persone curiose. Le competenze che costruiamo non sono competenze da programmatore: sono competenze da **regista di un lavoro fatto da altri**. Quegli altri, in questo caso, sono sistemi di intelligenza artificiale capaci di agire.

— Cosa impareremo, in una frase

Impareremo a passare da un'intelligenza artificiale che **risponde** a un'intelligenza artificiale che **fa**: che apre file, li scrive, li corregge, usa strumenti esterni, e alla fine ci riporta cosa ha fatto.

E impareremo la disciplina che rende tutto questo affidabile invece che caotico: il **context engineering**, cioè l'arte di decidere quali informazioni mettere davanti a un sistema di AI, in che ordine, e quali invece tenere fuori.

— Come è organizzata

La dispensa segue un percorso che va dalle fondamenta più concrete fino alle architetture più articolate. Ogni parte presuppone la precedente.

Parte I · Le fondamenta. L'ambiente di lavoro: gli accessi, il computer, i file, il testo, Markdown, il terminale, la condivisione del codice. È la parte meno "intelligenza

artificiale" e la più importante: senza queste basi, tutto il resto diventa magia incomprensibile.

Parte II · Come ragiona un agente. Che cos'è davvero un agente, in cosa differisce da un chatbot, qual è il ciclo che ripete sempre, e perché il contesto che gli diamo cambia il risultato.

Parte III · L'agente al lavoro. Installiamo e usiamo Claude Code, un agente che vive nel terminale. Impariamo a dargli una cartella, delle regole scritte e dei compiti verificabili.

Parte IV · Dare le mani all'agente: MCP. Colleghiamo l'agente al mondo esterno — posta, calendario, documenti — e affrontiamo il tema della sicurezza, che qui diventa serio.

Parte V · Ecosistema agentic. Da un agente solo a una piccola squadra: chi coordina, chi esegue, chi verifica.

Appendici. Tutto il materiale di consultazione rapida: comandi, prompt pronti, template, esercizi, glossario.

— Le convenzioni di lettura

Nel testo trovi alcuni elementi ricorrenti. Conviene riconoscerli subito.

I **comandi da digitare** e i **nomi di file** appaiono così: `pwd`, `README.md`. Vanno scritti esattamente come sono, compresi punti e trattini.

I **blocchi** su sfondo separato contengono testo da copiare integralmente: comandi, esempi di file, prompt da dare a un agente.

Poi ci sono quattro tipi di riquadro.

DA RICORDARE

Il concetto centrale del paragrafo, ridotto all'osso. Se ricordi solo questi, hai preso l'essenziale.

APPROFONDIMENTO

Materiale in più per chi vuole capire meglio o andare oltre. Si può saltare senza perdere il filo: non serve per fare gli esercizi.

ATTENZIONE

Errori tipici, trappole, cose che possono fare danni veri. Questi conviene leggerli.

FATTO QUANDO

Il criterio concreto per sapere se hai completato un passaggio. Non "ho capito", ma "ho questo risultato davanti".

– Windows e Mac

Ogni istruzione operativa è data in doppia versione, Windows e Mac, perché le differenze all'inizio bloccano più della teoria. Dove non è indicato nulla, l'istruzione vale per entrambi.

Se usi Linux, nella quasi totalità dei casi puoi seguire le istruzioni per Mac.

– Il metodo di lavoro

Tre abitudini che valgono per tutto il corso, e che vale la pena adottare da subito.

Chiedere aiuto è parte del metodo. Non è un fallimento, è la procedura normale. Nessuna domanda è banale: le domande "banali" sono quasi sempre quelle che bloccano metà dell'aula in silenzio.

Dire "fatto" o "bloccato". Durante gli esercizi, scrivere una di queste due parole nel canale comune permette a chi insegna di vedere a colpo d'occhio dove si trova il gruppo. "Bloccato" non è una resa: è un'informazione utile.

Tenere un diario di bordo. Alla fine di ogni sessione, tre righe in un file di testo: una cosa che ho capito, una cosa che mi ha bloccato, una cosa che voglio riprovare. Sembra un compito inutile. È lo strumento che fa la differenza fra aver seguito un corso e averlo imparato.

DA RICORDARE

Questa dispensa non serve a farti diventare programmatore. Serve a farti diventare qualcuno che sa dare istruzioni chiare a un sistema che esegue, e che sa controllare quello che ha eseguito.



Le fondamenta

Prima di parlare agli agenti bisogna sistemare il posto in cui lavoreremo. Questa parte non contiene intelligenza artificiale: contiene le fondamenta senza le quali l'intelligenza artificiale diventa una scatola nera che ogni tanto funziona e ogni tanto no.

L'idea di fondo è semplice: **il primo contesto da costruire siamo noi**. Un agente lavora dentro cartelle, file e comandi. Se quelle cose per noi sono nebbia, non potremo né guidarlo né controllarlo.

CAPITOLO 01

Account e accessi

Il corso poggia su tre accessi. Non sono tre account qualsiasi: ognuno copre una funzione diversa e insostituibile.

1.1 I tre accessi e a cosa servono

Il canale comune è il luogo dove la classe vive fra una lezione e l'altra. Nel corso usiamo Discord. Qui trovi annunci, materiali, schede degli esercizi; qui scrivi quando sei bloccato; qui incolli i link a quello che produci.

GitHub è la piattaforma dove vive il codice del mondo. Ci serve per due motivi: per condividere frammenti di codice in modo pulito (tramite un servizio collegato che si chiama Gist) e per vedere come sono fatti i progetti reali su cui gli agenti lavorano.

Claude è l'account con cui parleremo agli agenti. Lo useremo prima dal browser, come chatbot, e poi da terminale con Claude Code.

FATTO QUANDO

Sei entrato davvero in tutti e tre, hai scritto almeno un messaggio nel canale comune, e non stai dicendo "dovrebbe funzionare".

1.2 Il canale comune

Il canale comune serve a tenere insieme un gruppo che si vede a intervalli. Vale la pena capire come si usa, perché un canale usato male diventa rumore.

Le lezioni si svolgono in videochiamata; il canale scritto le accompagna. **La chat scritta è un canale di pari valore, non un ripiego**: nessuno è obbligato ad accendere microfono o telecamera, e scrivere va benissimo.

Durante gli esercizi vale la regola del *fatto o bloccato*: scrivi una delle due parole, così chi insegna vede a colpo d'occhio a che punto è il gruppo e raggiunge chi ha bisogno.

Un'ultima regola, importante: **il canale è la piazza, non l'archivio**. Il codice non si incolla lì dentro: si pubblica altrove e nel canale si incolla il link. Il perché è spiegato per esteso nel capitolo 7.

1.3 GitHub, in due parole

GitHub è una piattaforma per ospitare e collaborare su codice, costruita attorno a **Git**, il sistema di controllo di versione creato da Linus Torvalds. Fa essenzialmente tre cose.

Ospita i repository. Un repository (in gergo, *repo*) è una cartella di progetto che porta con sé l'intera storia delle proprie modifiche. GitHub la conserva online, raggiungibile da qualsiasi computer.

Traccia le versioni. Ogni modifica salvata è registrata come un *commit*. Si può tornare indietro nel tempo, vedere chi ha cambiato cosa e quando, e lavorare su linee parallele (*branch*) senza rompere il lavoro principale.

Abilita la collaborazione. Più persone lavorano sullo stesso progetto tramite *pull request* (proposte di modifica da rivedere e approvare), *issue* (segnalazioni e discussioni) e revisioni del codice.

Perché ci riguarda: GitHub è il posto dove gli strumenti di sviluppo assistito dall'AI si integrano in modo nativo. Il flusso tipico è: si scarica un repository, l'agente lavora su un ramo separato, apre una proposta di modifica, un umano la rivede. Conoscere questo flusso è il presupposto del lavoro agentic, anche per chi non scriverà mai una riga di codice.

APPROFONDIMENTO

Git e GitHub non sono la stessa cosa, ed è una confusione diffusa. Git è un programma che gira sul tuo computer e tiene la storia delle modifiche. GitHub è un'azienda che offre un servizio online costruito attorno a Git. Esistono alternative a GitHub (GitLab, Bitbucket, Codeberg) che usano lo stesso Git sottostante. In questo corso non useremo Git direttamente: ci basta capire il modello mentale.

1.4 Consigli pratici sugli account

Qualche accortezza che evita problemi nelle settimane successive.

Usa un indirizzo email che continuerai a controllare. Un account creato con un indirizzo temporaneo diventa irrecuperabile quando dovrai reimpostare la password.

Attiva l'autenticazione a due fattori su GitHub. È obbligatoria per molte operazioni e prima o poi te la chiederà comunque. Farlo con calma adesso è meglio che farlo di fretta durante un esercizio.

Annota da qualche parte quale email hai usato per quale servizio. Sembra banale. Non lo è: metà dei blocchi di accesso nasce da qui.

ATTENZIONE

Non salvare password in file di testo dentro le cartelle di progetto. Nel capitolo 7 vedremo perché questa è una delle abitudini più pericolose in assoluto quando si lavora con agenti e servizi collegati.

CAPITOLO 02

Orientarsi nel computer

Un agente lavora dentro una cartella. Se non sappiamo dire con precisione dove si trova un file, non possiamo dire a un agente dove lavorare, e non possiamo verificare cosa ha combinato. Questo capitolo copre il minimo indispensabile, ma lo copre davvero.

2.1 File, cartelle, percorsi

Un **file** è un contenitore di informazioni con un nome. Un **cartella** (o *directory*) è un contenitore di file e di altre cartelle. Le cartelle si annidano una dentro l'altra come scatole.

Un **percorso** (in inglese *path*) è l'indirizzo di un file: dice esattamente in quale sequenza di cartelle si trova. È come un indirizzo postale: via, numero civico, interno.

Ecco lo stesso concetto visto in due modi. Prima come struttura ad albero:

```
Documenti/  
  corso-ai/  
    laboratorio/  
      readme.md
```

E poi come percorso scritto su una riga sola:

```
/Users/nome/Documenti/corso-ai/laboratorio/readme.md
```

Le due scritture dicono la stessa identica cosa. La prima è più leggibile per noi, la seconda è quella che si scrive nel terminale e si dà agli strumenti.

2.2 La differenza fra Windows e Mac

I due sistemi scrivono i percorsi in modo diverso, ed è utile riconoscerli a colpo d'occhio.

Su **Mac e Linux** i percorsi usano la barra normale e partono dalla radice del disco:

```
/Users/nome/Documenti/corso-ai/laboratorio/readme.md
```

Su **Windows** i percorsi usano la barra rovesciata e partono da una lettera di unità:

```
C:\Users\nome\Documents\corso-ai\laboratorio\readme.md
```

C'è poi un'abbreviazione utilissima che incontrerai spesso: la **tilde** `~`. Indica la tua cartella personale, quella che contiene Scrivania, Documenti, Download. Quindi `~/Desktop` significa "la Scrivania dell'utente attuale", qualunque sia il suo nome.

APPROFONDIMENTO

Su Windows la tilde funziona in PowerShell ma non nel vecchio Prompt dei comandi. È uno dei tanti motivi per cui in questo corso, su Windows, useremo PowerShell e non CMD.

2.3 Come si scopre dove si trova un file

Capita spesso di avere un file davanti agli occhi ma non saper dire dove sia. Ecco come si scopre.

Su **Windows**: fai clic destro sul file, scegli *Proprietà*, e leggi la voce *Percorso*. In alternativa, con il file selezionato, la barra in alto della finestra mostra la posizione.

Su **Mac**: fai clic destro sul file, tieni premuto il tasto *Alt* (⌘) e scegli *Copia come percorso*. Il percorso completo finisce negli appunti, pronto da incollare.

FATTO QUANDO

Sai aprire la tua cartella di lavoro e sai dire ad alta voce dove si trova, senza esitare.

2.4 La trappola numero uno: le estensioni nascoste

L'**estensione** è la parte del nome che viene dopo l'ultimo punto: `.txt`, `.md`, `.pdf`, `.json`. Dice che tipo di file è quello.

Il problema è che sia Windows sia Mac, di serie, **nascondono le estensioni**. Così un file chiamato `relazione.txt` appare semplicemente come `relazione`, e da quel momento non si capisce più nulla: non distingui `note.txt` da `note.md`, non ti accorgi che hai salvato `README.md.txt` invece di `README.md`, e passi mezz'ora a chiederti perché l'agente non trova un file che tu vedi benissimo.

Attiviamole subito.

Su **Windows**: apri Esplora file, vai sulla scheda *Visualizza*, poi *Mostra*, e metti la spunta su *Estensioni nomi file*.

Su **Mac**: apri il Finder, menu *Finder*, poi *Impostazioni*, scheda *Avanzate*, e metti la spunta su *Mostra tutte le estensioni dei nomi dei file*.

ATTENZIONE

Questo è il singolo accorgimento che, da solo, previene la maggior parte dei problemi confusi delle prime settimane. Fallo adesso, non "dopo".

APPROFONDIMENTO

Perché i sistemi le nascondono? Per una scelta di design degli anni Novanta: si riteneva che l'utente non dovesse occuparsi del formato, ma solo dell'icona. Il risultato collaterale è stato un enorme vettore di truffe: un file chiamato `fattura.pdf.exe` appare come `fattura.pdf` con l'icona sbagliata, e molti ci cascano. Rendere visibili le estensioni è anche una misura di sicurezza.

2.5 Dove conviene tenere il lavoro

Un consiglio che sembra pedanteria e invece risolve problemi veri: **crea una cartella unica per il corso e tienici dentro tutto.**

Per esempio `corso-ai` sulla Scrivania, e dentro una sottocartella per ogni esercizio. Evita di lavorare direttamente sulla Scrivania in mezzo ad altri cinquanta file, e soprattutto evita nomi con spazi e accenti: `il mio progetto finale` prima o poi ti darà problemi nel terminale, mentre `progetto-finale` non te ne darà mai.

La regola pratica per i nomi di file e cartelle:

- solo lettere minuscole, numeri, trattini `-` e trattini bassi `_`;
- niente spazi, niente accenti, niente `& / \ : * ? " < > |`;
- nomi che descrivono il contenuto, non `documento1` e `documento1-def-vero`.

DA RICORDARE

Una cartella ordinata non è pignoleria estetica. Quando arriveremo agli agenti, vedremo che **la cartella è il contesto**: se è disordinata, l'agente parte già male.

CAPITOLO 03

Il testo puro

Tutto quello che faremo nel corso — istruzioni per gli agenti, documentazione, configurazioni, codice — è testo puro. Vale la pena capire cosa significa esattamente, perché è meno ovvio di quanto sembri.

3.1 Cosa vuol dire "puro"

Un file di **testo puro** contiene soltanto caratteri: lettere, numeri, punteggiatura, a capo. Niente altro.

Un documento Word, al confronto, contiene i caratteri **più** una quantità di informazioni invisibili: quale font, quale dimensione, quali margini, dove va l'immagine, chi ha fatto quale modifica. Sono informazioni utili per stampare una lettera, e completamente inutili — anzi, dannose — quando il destinatario è un programma.

Il testo puro ha tre proprietà che ce lo rendono indispensabile:

- **si apre ovunque**, oggi e fra vent'anni, con qualsiasi programma;
- **non nasconde nulla**: quello che vedi è tutto quello che c'è;
- **è confrontabile**: si può dire con precisione quali righe sono cambiate fra due versioni. Su questa proprietà è costruito tutto il controllo di versione.

ATTENZIONE

Non scrivere mai file di codice o di istruzioni con Word o Google Docs. Questi programmi trasformano automaticamente le virgolette dritte " in virgolette tipografiche " ", e i trattini in trattini lunghi. Il risultato è codice che *sembra* giusto e non funziona, per un motivo praticamente invisibile a occhio nudo.

3.2 Installare un editor di testo

Ci serve un programma pensato apposta per il testo puro. Nel corso usiamo **Sublime Text**, che è leggero, veloce e gratuito da provare senza limiti di tempo.

Come si installa:

- 01 Vai sul sito ufficiale sublimetext.com.
- 02 Scarica la versione per il tuo sistema operativo (il sito di solito la riconosce da solo).
- 03 Avvia il file scaricato e segui la procedura guidata.
- 04 Accetta i termini di licenza e conferma il percorso di installazione proposto.
- 05 Apri il programma.

FATTO QUANDO

Hai aperto l'editor e vedi una finestra bianca vuota pronta per scrivere.

APPROFONDIMENTO

Sublime Text non è l'unica scelta. **Visual Studio Code** (gratuito, di Microsoft) è oggi lo standard più diffuso: è più pesante ma include un terminale integrato, l'anteprima di Markdown e l'integrazione con Git. Se pensi di continuare a lavorare in quest'area dopo il corso, prima o poi ci passerai. Su Mac esiste anche **BEdit**; su Windows **Notepad++**. Tutti vanno bene: l'importante è che siano editor di testo, non elaboratori di documenti.

3.3 Il primo file

Facciamo subito la prova, perché è meno banale di quanto sembri.

- 01 Nell'editor, crea un nuovo file (menu *File*, poi *Nuovo*).
- 02 Scrivi due o tre righe qualsiasi.
- 03 Salva (*File*, poi *Salva con nome*).
- 04 **Nel nome, scrivi anche l'estensione:** `appunti.txt`, non solo `appunti`.
- 05 Salvalo nella cartella del corso che hai creato prima.

Ora riapri quel file con un programma diverso — il Blocco note su Windows, TextEdit su Mac — e osserva che il contenuto è esattamente lo stesso. Questa è la proprietà che ci interessa.

Questo è testo puro.
Solo caratteri, niente formattazione nascosta.

APPROFONDIMENTO

Se vuoi vedere l'unica cosa "invisibile" che un file di testo contiene davvero, cerca *codifica dei caratteri* e *UTF-8*. In breve: il computer memorizza numeri, e la codifica è la tabella che dice quale numero corrisponde a quale carattere. UTF-8 è lo standard universale odierno e gestisce accenti, emoji e alfabeti non latini. Quando apri un file e vedi `perchÃ` al posto di `perché`, è quasi sempre un problema di codifica: il file è stato salvato con una tabella e letto con un'altra.

CAPITOLO 04

La famiglia dei formati

I file di testo puro non sono tutti uguali. L'estensione dice come è organizzato il testo al loro interno, e quindi chi lo sa leggere. Vale la pena conoscere i sei che si incontrano più spesso.

4.1 I sei formati da riconoscere

`.txt` — testo senza struttura. Solo righe di testo. Nessuna regola, nessuna gerarchia. Perfetto per appunti volanti, inadatto a tutto il resto.

`.md` — Markdown: struttura leggera, per gli umani. Aggiunge al testo puro pochi simboli che indicano titoli, elenchi, grassetti. Resta perfettamente leggibile anche senza essere formattato. È il formato principale di questo corso e ha un capitolo tutto suo.

`.csv` — dati a tabella. Righe e colonne separate da virgole. Una riga per record, un campo per colonna. È il formato con cui i fogli di calcolo si scambiano dati.

```
nome,ruolo,anni
Anna,coordinatrice,7
Luca,esecutore,3
```

`.json` — dati strutturati, per le macchine. Organizza le informazioni in coppie *nome: valore* e permette di annidare strutture una dentro l'altra. È il formato in cui i programmi si parlano fra loro, ed è anche il formato in cui gli strumenti degli agenti scambiano dati. Il secondo formato importante del corso.

```
{
  "progetto": "manuale",
  "lingua": "italiano",
  "capitoli": 6,
  "autori": ["Anna", "Luca"]
}
```

`.yaml` — dati e configurazioni, leggibili. Dice le stesse cose del JSON ma con meno punteggiatura, usando l'indentazione al posto delle parentesi. Si usa moltissimo nei file di configurazione. Lo incontreremo nella Parte V, in testa ai file che definiscono i singoli agenti.

```
progetto: manuale
lingua: italiano
capitoli: 6
```

`.xml` — il parente più anziano. Struttura basata su etichette aperte e chiuse, come l'HTML. Oggi è meno usato per i dati nuovi, ma spunta ancora spesso: i file Word, per esempio, sono XML compresso.

4.2 Su cosa ci concentriamo

Di questi sei, il corso ne usa intensamente due: **Markdown** e **JSON**.

Markdown è la lingua in cui scriveremo agli agenti: istruzioni, documentazione, regole di progetto. JSON è la lingua in cui gli agenti e i loro strumenti si scambiano dati fra loro; noi lo leggeremo più che scriverlo, ma saperlo riconoscere aiuta a capire cosa sta succedendo quando un agente usa uno strumento esterno.

DA RICORDARE

Non serve saper scrivere tutti i formati. Serve saperli **riconoscere**: guardare un file e dire "questo è JSON, quindi contiene dati per un programma" invece di "questo è un guazzabuglio di parentesi".

APPROFONDIMENTO

Un errore molto comune con JSON: la virgola dopo l'ultimo elemento di una lista. `["a", "b", "c",]` è **sbagliato** e il file viene rifiutato. Molti linguaggi la tollerano, JSON no. Se un agente ti segnala un errore di *parsing* su un file JSON, quella virgola è il primo posto dove guardare, insieme alle virgolette non chiuse.

CAPITOLO 05

Markdown

Markdown è il formato più importante di questa dispensa. Non perché sia complicato — si impara in mezz'ora — ma perché è **la lingua in cui parleremo agli agenti**.

5.1 Cos'è e perché ci serve

Markdown è un linguaggio di *markup leggero*: un modo per dare struttura al testo usando pochi simboli semplici, restando leggibile anche prima di essere formattato.

La differenza con Word è concettuale. In Word la formattazione si applica con pulsanti e menu, e finisce dentro il file in forma invisibile. In Markdown **la struttura è visibile nel testo stesso**: un titolo si riconosce perché ha un cancelletto davanti, sia quando lo scrivi sia quando lo rileggi fra sei mesi.

Per questo Markdown è diventato lo standard della documentazione tecnica, dei repository su GitHub, delle wiki, dei blog e — più di recente — dei sistemi basati su intelligenza artificiale.

Lo useremo per scrivere:

- **README.md**, il documento che descrive un progetto;
- **AGENTS.md** e **CLAUDE.md**, i file di regole che guidano gli agenti;
- documentazione di progetto;
- appunti tecnici e guide operative.

DA RICORDARE

Il modo in cui organizziamo le informazioni cambia la qualità delle risposte che otteniamo. Scrivere in Markdown non è decorazione: **è già context engineering**.

5.2 Titoli

I titoli si creano con il cancelletto `#`. Più cancelletti metti, più il livello scende nella gerarchia.

```
# Titolo principale

## Sezione

### Sottosezione
```

Serve uno spazio dopo i cancelletti: `#Titolo` non funziona, `# Titolo` sì.

I titoli non servono solo a fare bella figura. Servono a **far capire la struttura del documento**, sia a una persona che lo apre sia a un agente che lo legge. Un file di istruzioni ben titolato viene seguito meglio di uno scritto tutto di seguito.

5.3 Grassetto e corsivo

```
**testo in grassetto**

*testo in corsivo*
```

Il risultato è **testo in grassetto** e *testo in corsivo*.

Regola pratica: usa il grassetto per le poche cose che devono saltare all'occhio. Se grassetti tutto, non è grassetto niente.

5.4 Elenchi

Gli elenchi puntati si fanno con un trattino a inizio riga:

```
- primo elemento
- secondo elemento
- terzo elemento
```

Quelli numerati con un numero seguito da punto:

```
1. primo passaggio
2. secondo passaggio
3. terzo passaggio
```

Gli elenchi sono particolarmente utili quando descrivi requisiti, materiali, sequenze di operazioni. Quando scriverai regole per gli agenti, li userai continuamente.

APPROFONDIMENTO

Nei numerati puoi scrivere **1.** su tutte le righe: quasi tutti i programmi rinumerano da soli in fase di visualizzazione. Comodo quando devi inserire un passaggio in mezzo senza rifare tutta la numerazione a mano.

5.5 Link

```
[Testo del link](https://esempio.it)
```

Le parentesi quadre contengono il testo che si vede, le tonde l'indirizzo. Fra la quadra chiusa e la tonda aperta non ci va nessuno spazio: è l'errore più frequente.

I link servono a collegare documentazione, repository, articoli, risorse. In un file di istruzioni per un agente, servono anche a indicargli dove andare a leggere.

5.6 Codice dentro una frase

Quando citi un comando, un nome di file o una variabile all'interno di un discorso, si usano gli apici inversi, detti *backtick*:

Apri il file ``README.md`` e verifica il comando ``npm install``.

Il risultato è: apri il file `README.md` e verifica il comando `npm install`.

Il vantaggio non è estetico: il testo dentro i backtick viene mostrato con un carattere a spaziatura fissa e **non subisce sostituzioni automatiche**. È il modo giusto per scrivere qualcosa che deve essere copiato esattamente.

ATTENZIONE

Il backtick non è l'apostrofo. Su tastiera italiana Windows si ottiene con `Alt Gr` + il tasto dell'apostrofo, oppure con `Alt + 96` dal tastierino numerico. Su Mac italiano con `Alt + ``. È il carattere che sta anche sotto il tasto della `è` maiuscola su molte tastiere. Se stai usando `'` o `´`, non funzionerà.

5.7 Blocchi di codice

Per frammenti più lunghi si aprono e si chiudono **tre backtick**. Dopo i primi tre si può scrivere il nome del linguaggio, così l'editor colora il testo di conseguenza.

Si scrive così:

```
```javascript
console.log("Hello World");
```
```

E il risultato è un blocco separato, con il codice ben distinto dal testo intorno.

I linguaggi che indicherai più spesso: `bash` per i comandi di terminale, `json`, `yaml`, `markdown`, `text` quando non è codice ma vuoi comunque un blocco separato.

DA RICORDARE

Il blocco di codice è l'elemento Markdown che userai di più in assoluto. Ogni volta che condividi un comando, un errore o un frammento, va dentro tre backtick.

5.8 Tabelle

Le tabelle si costruiscono con le barre verticali. La seconda riga, quella con i trattini, è obbligatoria e separa l'intestazione dal contenuto.

| Comando | Descrizione | Esempio |
|---------|-------------------------|---------------------------|
| ----- | ----- | ----- |
| Prompt | Istruzione data all'AI | "Scrivi una funzione..." |
| Context | Informazioni aggiuntive | README.md, documentazione |
| Output | Risultato prodotto | Codice, testo, immagini |

Non serve che le colonne siano allineate perfettamente nel testo sorgente: il risultato finale è identico. Allinearle serve solo a te, per rileggerle.

5.9 Un README completo

Mettiamo tutto insieme. Questo è un esempio realistico, coerente con il tema del corso, che puoi usare come modello.

```
# AI Coding Agent Project

Questo progetto esplora l'uso degli agenti di intelligenza artificiale
per la scrittura, la modifica e la comprensione del codice.

## Obiettivi

- comprendere il funzionamento degli agenti AI
- imparare a scrivere istruzioni efficaci
- documentare un progetto con Markdown
- costruire un piccolo prototipo funzionante

## Strumenti

- GitHub
- GitHub Gist
- Markdown
- Editor di codice
- Agenti AI

## Istruzione di esempio

```text
Analizza questo progetto.
Identifica i problemi principali.
Proponi miglioramenti progressivi senza riscrivere completamente il codice.
```

## Note

Questo progetto fa parte di un percorso dedicato ad agentic coding
e context engineering.
```

Nota la struttura: un titolo principale, una descrizione breve, poi sezioni di secondo livello. È lo schema che ritroverai in praticamente ogni progetto serio.

5.10 Esercizio

Consegna. Crea un file chiamato `README.md` che contenga, in quest'ordine:

- un titolo;
- una breve descrizione di te o del progetto;
- un elenco di esperienze pregresse o di obiettivi;
- un elenco di strumenti;
- un blocco di codice contenente un'istruzione per un agente AI;
- un link a una risorsa esterna.

FATTO QUANDO

Hai prodotto e salvato il file, l'estensione è davvero `.md` e non `.md.txt`, e riaprendolo riconosci la struttura che hai scritto.

APPROFONDIMENTO

Markdown esiste in varianti leggermente diverse (*dialetti*). Quella che incontrerai quasi sempre è **GitHub Flavored Markdown**, che aggiunge tabelle, caselle di spunta - `[ ]` e barrato `~~così~~`. Per vedere l'anteprima mentre scrivi: su Visual Studio Code esiste un pulsante di anteprima integrato; in alternativa qualunque editor Markdown online mostra il risultato affiancato al sorgente.

CAPITOLO 06

Il terminale

Il terminale spaventa più di quanto meriti. È solo un modo per dare comandi al computer scrivendo, invece che cliccando.

Ci serve per installare strumenti, avviare programmi e, soprattutto, perché **è la stanza in cui vive l'agente** di cui ci occuperemo nella Parte III. Non dobbiamo diventare esperti: dobbiamo saperci muovere fra le cartelle e lanciare un comando senza paura.

6.1 Come si apre

Su **Windows**: premi il tasto Windows, scrivi `PowerShell`, e apri *Windows PowerShell*.

Su **Mac**: premi `Cmd + Spazio`, scrivi `Terminale`, e premi Invio.

Quello che vedi è una finestra con una riga di testo che aspetta. Quella riga si chiama **prompt** — attenzione, stesso nome ma niente a che vedere con i prompt che daremo all'AI — e di solito mostra dove ti trovi in questo momento.

ATTENZIONE

Su Windows esistono due terminali diversi: il vecchio *Prompt dei comandi* (CMD) e *PowerShell*. Hanno sintassi diverse e comandi che si comportano diversamente. **In questo corso usiamo sempre PowerShell.** Lo riconosci perché la riga di solito comincia con `PS`. Se un comando dà un errore incomprensibile, la prima cosa da controllare è di non essere finito in CMD.

6.2 La regola d'oro: sapere sempre dove sei

Il terminale, in ogni momento, è "dentro" una cartella. Tutti i comandi che dai si riferiscono a quella cartella. È il concetto singolo più importante di questo capitolo.

Il comando che risponde alla domanda "dove sono?" è:

```
pwd
```

Sta per *print working directory*, cioè "stampa la cartella di lavoro". Funziona su Mac, Linux e PowerShell.

Prendi l'abitudine di lanciarlo ogni volta che hai un dubbio. Quando arriveremo agli agenti, "non sapere in quale cartella sei" sarà la causa numero uno dei problemi.

6.3 I comandi essenziali

| Azione | Mac / Linux | Windows (PowerShell) |
|---------------|------------------|----------------------|
| Dove mi trovo | <code>pwd</code> | <code>pwd</code> |

| Azione | Mac / Linux | Windows (PowerShell) |
|--------------------------------------|-----------------------------|---|
| Elenco dei file | <code>ls</code> | <code>ls</code> oppure <code>dir</code> |
| Elenco completo, anche file nascosti | <code>ls -la</code> | <code>ls -Force</code> |
| Entrare in una cartella | <code>cd nome</code> | <code>cd nome</code> |
| Tornare alla cartella superiore | <code>cd ..</code> | <code>cd ..</code> |
| Andare alla cartella personale | <code>cd ~</code> | <code>cd ~</code> |
| Creare una cartella | <code>mkdir progetto</code> | <code>mkdir progetto</code> |
| Creare un file vuoto | <code>touch nome.md</code> | <code>New-Item nome.md</code> |
| Pulire lo schermo | <code>clear</code> | <code>cls</code> |
| Chi sono | <code>whoami</code> | <code>whoami</code> |
| Rimuovere un file | <code>rm file.txt</code> | <code>del file.txt</code> |
| Rimuovere una cartella vuota | <code>rmdir cartella</code> | <code>rmdir cartella</code> |
| Verificare Node.js | <code>node -v</code> | <code>node -v</code> |
| Verificare npm | <code>npm -v</code> | <code>npm -v</code> |

6.4 Tre trucchi che cambiano la vita

Questi tre non sono comandi, sono abitudini. Fanno la differenza fra digitare a fatica e muoversi con scioltezza.

Il tasto Tab completa i nomi. Scrivi `cd Doc` e premi Tab: il terminale completa da solo in `cd Documenti`. Oltre a farti risparmiare tempo, evita gli errori di battitura — e se Tab *non* completa, vuol dire che quella cartella lì dentro non c'è, il che è già un'informazione preziosa.

La freccia in su richiama i comandi precedenti. Premila più volte per scorrere la cronologia. Serve moltissimo quando devi ripetere un comando lungo con una piccola variazione.

Ctrl + C interrompe. Se un comando è partito e non finisce più, o se hai scritto qualcosa di sbagliato e vuoi ricominciare la riga, `Ctrl + C` ti riporta al prompt. Vale anche su Mac (`Ctrl`, non `Cmd`).

6.5 Percorsi con gli spazi

Se una cartella si chiama `il mio progetto`, il comando `cd il mio progetto` fallisce: il terminale legge tre parole separate. Va scritto fra virgolette:

```
cd "il mio progetto"
```

Ecco perché il capitolo 2 consigliava di evitare gli spazi nei nomi.

6.6 Un primo giro completo

Proviamo la sequenza che useremo decine di volte. Riga per riga, leggendo cosa succede.

```
cd ~/Desktop
mkdir laboratorio
cd laboratorio
pwd
ls
```

In ordine: vai sulla Scrivania; crea lì una cartella chiamata **laboratorio**; entra dentro quella cartella; verifica dove sei; guarda cosa c'è dentro (per ora, niente).

FATTO QUANDO

Hai aperto il terminale, lanciato questa sequenza senza errori, e **pwd** ti mostra un percorso che finisce con **laboratorio**.

ATTENZIONE

Il comando **rm** **non manda nel cestino: cancella**. Non c'è nessuna finestra di conferma e non c'è modo di annullare. Trattalo con rispetto, e diffida di qualunque comando trovato online che contenga **rm -rf** senza che tu abbia capito esattamente cosa cancella.

APPROFONDIMENTO

Se usi Windows e ti capiterà di seguire tutorial scritti per Mac o Linux, prima o poi incontrerai **WSL** (*Windows Subsystem for Linux*): un modo per avere un vero terminale Linux dentro Windows. Non serve per questo corso, ma è la strada naturale se vorrai proseguire, perché elimina d'un colpo tutte le differenze di sintassi fra i due mondi.

CAPITOLO 07

Condividere codice ed errori

Ultimo capitolo delle fondamenta, e uno dei più utili nella pratica quotidiana. Riguarda due gesti che farai continuamente: mandare a qualcuno un pezzo di codice, e mandare a qualcuno un errore.

7.1 Perché il codice non si incolla in chat

Quando incolli codice dentro una chat — WhatsApp, la chat di una videochiamata, un'email — succedono tre cose che lo rovinano.

L'indentazione salta. Gli spazi a inizio riga, che in molti linguaggi hanno un significato preciso, vengono appiattiti o cancellati.

Le righe lunghe vanno a capo nei punti sbagliati, spezzando istruzioni che dovevano stare unite.

Alcuni caratteri vengono sostituiti in automatico. Le virgolette dritte diventano virgolette "abbellite", i trattini diventano trattini lunghi. Il computer non li riconosce più.

Il risultato è codice che sembra giusto e non funziona, con un motivo invisibile a occhio nudo. È una perdita di tempo per chi lo manda e per chi lo riceve.

La soluzione è usare uno strumento fatto apposta. Nel corso usiamo **GitHub Gist**.

7.2 Cos'è un Gist

GitHub Gist è un servizio gratuito per pubblicare rapidamente frammenti di codice, esempi, configurazioni o porzioni di testo, senza dover creare un progetto completo.

Ogni frammento pubblicato diventa **una pagina con un indirizzo tutto suo**, che si condivide con un semplice link. In più ogni Gist conserva lo storico delle modifiche: si può aggiornare un esempio e vedere cosa è cambiato fra una versione e l'altra.

7.3 Come si crea, passo per passo

- 01 Vai su gist.github.com e accedi con il tuo account GitHub.
- 02 Nel campo in alto, scrivi un nome per il file, per esempio `esempio.py` oppure `note.md`. **L'estensione conta:** è quella che dice a Gist come colorare il codice.
- 03 Nel riquadro grande sotto, incolla il contenuto.
- 04 In basso a destra scegli come pubblicarlo. *Create secret gist* crea un frammento raggiungibile solo da chi ha il link, ed è quello che useremo di solito. *Create public gist* lo rende visibile e ricercabile da chiunque.
- 05 Dopo la creazione, copia l'indirizzo dalla barra del browser e incollalo nel canale comune. Quello è il link da condividere.

Per aggiornare un frammento già pubblicato: riapri la sua pagina, premi *Edit*, modifica e salva. Il link resta lo stesso e la versione precedente rimane nello storico.

APPROFONDIMENTO

"Secret" non significa "protetto da password". Un Gist segreto non compare nelle ricerche e non è elencato sul tuo profilo, ma chiunque abbia il link può aprirlo. È riservatezza, non sicurezza. Se un contenuto è davvero sensibile, non va su Gist e basta.

Se Gist dovesse dare problemi, l'alternativa è **Pastebin** (pastebin.com), che funziona in modo simile: incolli il testo, premi *Create New Paste*, condividi il link che ottieni.

7.4 La regola non opzionale sulle chiavi

Questa è la regola più importante del capitolo, e forse dell'intera Parte I.

ATTENZIONE

Non si incolla mai una chiave, una password o un token dentro un Gist, un Pastebin o qualunque servizio di condivisione. Questi frammenti sono raggiungibili tramite un link e, se pubblici, sono anche ricercabili. Una chiave lasciata lì è come lasciare le chiavi di casa attaccate alla porta.

Una **chiave di accesso** (o *token*, o *API key*) è una stringa di caratteri che identifica te presso un servizio. Chi la possiede può agire a tuo nome: consumare il tuo credito, leggere i tuoi dati, mandare messaggi come te. In un corso sugli agenti le chiavi girano spesso, quindi il rischio è concreto e non teorico.

Prima di pubblicare qualsiasi cosa: rileggila e sostituisci ogni chiave con un segnaposto, per esempio `LA_TUA_CHIAVE QUI`.

Se ti accorgi di averne pubblicata una: cancellare il Gist **non basta**. Bisogna andare sul servizio che ha emesso quella chiave, **revocarla** (cioè disattivarla) e generarne una nuova. Cancellare non basta perché nel frattempo qualcuno potrebbe averla già copiata — e sul web esistono programmi automatici che fanno esattamente questo, a ciclo continuo, ventiquattr'ore al giorno.

7.5 Un errore si copia, non si fotografa

Quando qualcosa non funziona, il computer scrive un messaggio di errore. Quel messaggio è la prima cosa da guardare, ed è quasi sempre più informativo di quanto sembri.

La tentazione, all'inizio, è fotografare lo schermo con il telefono. È il modo peggiore. Un errore in formato testo si può **cercare** su un motore di ricerca, **incollare** a un agente perché lo interpreti, e **leggere** bene anche su uno schermo piccolo. Una fotografia no: non è cercabile, spesso è sfocata o storta, e costringe chi ti aiuta a ricopiare tutto a mano.

Come si copia il testo di un errore dal terminale:

Su **Windows**, seleziona le righe trascinando il mouse, poi **Ctrl + C**. In alcune configurazioni di PowerShell il tasto destro copia direttamente la selezione.

Su **Mac**, seleziona trascinando il mouse, poi **Cmd + C**.

Se l'errore compare in una finestra a comparsa, prova comunque a selezionarlo con il mouse: molte finestre permettono di copiare il testo anche quando non sembra.

Quando l'hai copiato, incollalo nel canale comune **dentro un blocco di codice**, così mantiene la sua forma.

7.6 Quando serve davvero un'immagine

A volte il problema non è testuale ma visivo: un pulsante che non trovi, una schermata che si comporta in modo strano. In quel caso si fa uno **screenshot pulito**, non una foto con il telefono.

Su **Windows**: premi **Windows + Maiusc + S**. Lo schermo si oscura leggermente e puoi selezionare col mouse solo la parte che ti interessa. L'immagine finisce negli appunti e la incolli con **Ctrl + V**.

Su **Mac**: premi **Cmd + Maiusc + 4**. Il puntatore diventa un mirino e puoi trascinare per selezionare l'area. L'immagine viene salvata sulla Scrivania, pronta da trascinare nel canale.

Il principio è: **cattura solo la parte che serve**, non tutto lo schermo, così chi guarda capisce subito dove guardare.

7.7 Come si chiede aiuto bene

Scrivere "non funziona" costringe chi ti aiuta a fare venti domande prima di poter fare qualcosa. Scrivere una mini-scheda del problema fa risparmiare tempo a tutti — e molto spesso, mentre la compili, ti accorgi da solo di dov'è il problema.

La scheda contiene cinque righe:

- 01 sistema operativo:** Mac, Windows o Linux;
- 02 la cartella in cui ti trovi:** il risultato di `pwd`;
- 03 il comando che hai lanciato**, copiato esattamente;
- 04 l'errore**, copiato esattamente;
- 05 cosa ti aspettavi che succedesse.**

DA RICORDARE

Il debugging comincia nel momento in cui smetti di dire "*non va*" e cominci a dire "*questo comando produce questo errore*". È la stessa disciplina che, più avanti, ti servirà per capire perché un agente ha fatto una cosa diversa da quella che volevi.

— Riepilogo della Parte I

A questo punto dovresti avere:

- i tre accessi funzionanti, verificati davvero;
- le estensioni dei file visibili;
- un editor di testo installato e una cartella di lavoro ordinata;
- il tuo primo file Markdown, salvato con l'estensione giusta;
- il terminale aperto almeno una volta, e la certezza di saper rispondere alla domanda "in quale cartella sono?";
- un Gist pubblicato, senza chiavi dentro.

Sono fondamenta poco spettacolari, e proprio per questo si tende a saltarle. Ma tutto quello che viene dopo — l'agente che scrive file, le regole persistenti, gli strumenti collegati, la squadra di agenti che si coordina — poggia esattamente qui sopra. Da qui in avanti, cominciamo a parlare di intelligenza artificiale.